main_commented.py

```
# ========== IMPORT STATEMENTS ==========
# Import required libraries
import json  # For reading/writing JSON data
import tkinter as tk  # For creating GUI windows and widgets
from tkinter import ttk, messagebox  # ttk for modern widgets, messagebox for dialogs
from pathlib import Path  # For working with file paths
import uuid  # For generating unique IDs for each book
# ========== GLOBAL VARIABLES ==========
# Define the path to the JSON file that stores book data
DATA_FILE = Path(__file__).with_name("media.json")
# ========== MAIN APPLICATION CLASS ==========
# Main application class for the library GUI
class LibraryApp:
    # Constructor to initialize the application window and components
    def __init__(self, root):
        # Store reference to root window
        self.root = root
        # Set window title
        self.root.title("Atharva Rakshe Library")
        # Set window size (width x height)
        self.root.geometry("900x600")
        # List to store all book data loaded from JSON
        self.data = []
        # Flag to track sorting order (True = ascending, False = descending)
        self.sort_ascending = True
        # ========== CONFIGURE TREEVIEW STYLE ==========
        # Create a style object to customize Treeview appearance
        style = ttk.Style()
        # Use the 'clam' theme for modern look
        style.theme_use('clam')
        # Configure Treeview row height and font
        style.configure("Treeview", rowheight=28, font=("Arial", 10))
        # Configure heading font to be bold
        style.configure("Treeview.Heading", font=("Arial", 11, "bold"))
        # Set background color for Treeview
        style.configure("Treeview", background="#F5F5F5", fieldbackground="#F5F5F5")
        # Set selection color (purple) when a row is selected
        style.map('Treeview', background=[('selected', '#5C4A99')])
        # ========== CREATE HEADER FRAME ==========
        # Create header frame with purple background at the top
        header = tk.Frame(root, bg="#5C4A99", height=80)
        # Pack at top, fill horizontally
        header.pack(side=tk.TOP, fill=tk.X)
        # Prevent shrinking to content size
        header.pack_propagate(False)
        # Create and display main title with book emoji
        title_label = tk.Label(header, text="■  Scholar's Digital Library", font=("Arial", 2
4, "bold"), fg="white", bg="#5C4A99")
        # Add padding
        title_label.pack(pady=(10, 0))

        # Create and display subtitle
        subtitle_label = tk.Label(header, text="Organize and manage your media collection",
font=("Arial", 10), fg="#E0E0E0", bg="#5C4A99")
        # Add padding
        subtitle_label.pack(pady=(0, 10))
        # ========== CREATE TOOLBAR ==========
        # Create top toolbar frame containing category, search, and action buttons
        top = ttk.Frame(root, padding=(8, 6))
        # Pack at top, fill horizontally
        top.pack(side=tk.TOP, fill=tk.X)
        # ===== CATEGORY FILTER =====
        # Category filter label
        ttk.Label(top, text="Category:").pack(side=tk.LEFT)
        # Variable to store selected category
        self.category_var = tk.StringVar()
        # Category dropdown combobox (read-only)
        self.category_cb = ttk.Combobox(top, textvariable=self.category_var, width=18, state
='readonly')
```

```
        # Pack on left side
        self.category_cb.pack(side=tk.LEFT, padx=(6, 8))
        # Bind selection event so choosing a category immediately filters the table
        self.category_cb.bind('<<ComboboxSelected>>', lambda e: self.on_category_change())
        # ===== LOAD BUTTON =====
        # Load button to reload data from JSON
        self.load_btn = ttk.Button(top, text="Load", command=self.on_load)
        # Pack on left side
        self.load_btn.pack(side=tk.LEFT)
        # ===== SEARCH FILTER =====
        # Search label
        ttk.Label(top, text="  Name:").pack(side=tk.LEFT, padx=(10, 4))
        # Variable to store search query
        self.search_var = tk.StringVar()
        # Search textbox for entering book name
        self.search_entry = ttk.Entry(top, textvariable=self.search_var, width=30)
        # Pack on left side
        self.search_entry.pack(side=tk.LEFT)

        # Search button to filter by book name
        self.search_btn = ttk.Button(top, text="Search", command=self.on_search)
        # Pack on left side with padding
        self.search_btn.pack(side=tk.LEFT, padx=(6, 8))
        # ========== ACTION BUTTONS (RIGHT SIDE) ==========
        # Erase button to delete selected book
        self.erase_btn = ttk.Button(top, text="Erase", command=self.on_erase)
        # Pack on right side
        self.erase_btn.pack(side=tk.RIGHT, padx=(6, 0))

        # New button to add a new book
        self.new_btn = ttk.Button(top, text="New", command=self.on_new)
        # Pack on right side
        self.new_btn.pack(side=tk.RIGHT, padx=(6, 0))
        # Sort button to toggle date sorting order
        self.sort_btn = ttk.Button(top, text="Sort by Date ↑", command=self.on_sort_toggle)
        # Pack on right side with more padding
        self.sort_btn.pack(side=tk.RIGHT, padx=(6, 14))
        # ========== CREATE TREEVIEW (TABLE) ==========
        # Container frame for treeview and scrollbar
        tree_container = tk.Frame(root)
        # Pack and expand to fill available space
        tree_container.pack(side=tk.TOP, fill=tk.BOTH, expand=True, padx=8, pady=(6, 8))
        # ===== TREEVIEW SETUP =====
        # Define columns for the table (include serial number)
        columns = ("sno", "name", "author", "date", "category")
        # Create the Treeview widget (table)
        self.tree = ttk.Treeview(tree_container, columns=columns, show='headings', style="Tr
eeview")

        # Set column headings (S.No first)
        self.tree.heading("sno", text="S.No")
        self.tree.heading("name", text="Name")
        self.tree.heading("author", text="Author")
        self.tree.heading("date", text="Date")
        self.tree.heading("category", text="Category")
        # Set column widths and alignment (all centered)
        self.tree.column("sno", anchor='center', width=60)
        self.tree.column("name", anchor='center', width=360)
        self.tree.column("author", anchor='center', width=160)
        self.tree.column("date", anchor='center', width=80)
        self.tree.column("category", anchor='center', width=120)
        # ========== CONFIGURE ROW COLORS ==========
        # Configure odd rows (white background)
        self.tree.tag_configure('oddrow', background='#FFFFFF')
        # Configure even rows (light blue background for alternating effect)
        self.tree.tag_configure('evenrow', background='#E8F4F8')
        # ========== ADD SCROLLBAR ==========
        # Create vertical scrollbar for the treeview
        vsb = ttk.Scrollbar(tree_container, orient=tk.VERTICAL, command=self.tree.yview)
        # Link scrollbar to treeview
        self.tree.configure(yscroll=vsb.set)
        # Pack scrollbar on the right
```

```
        vsb.pack(side=tk.RIGHT, fill=tk.Y)
        # Pack treeview on left, fill remaining space
        self.tree.pack(side=tk.TOP, fill=tk.BOTH, expand=True)
        # ========== EMPTY STATE LABEL ==========
        # Label displayed when no books are in the library
        self.empty_label = tk.Label(tree_container, text="Add book to the library", font=("A
rial", 14), fg="#999999")
        # Pack in center of container
        self.empty_label.pack(side=tk.TOP, fill=tk.BOTH, expand=True)
        # ========== CONTEXT MENU ==========
        # Create a context menu (right-click) with Edit and Cancel options
        self.context_menu = tk.Menu(self.root, tearoff=0)
        self.context_menu.add_command(label="Edit", command=self.on_edit_selected)
        self.context_menu.add_command(label="Cancel", command=lambda: None)
        # Bind right-click on the treeview to show the context menu
        self.tree.bind('<Button-3>', self.on_right_click)
        # ========== LOAD INITIAL DATA ==========
        # Load data from JSON file when app starts
        self.on_load()
    # ========== FILE OPERATIONS METHODS ==========
    # Method to read data from JSON file
    def read_json(self):
        # If file doesn't exist, return empty list
        if not DATA_FILE.exists():
            return []
        try:
            # Open and read JSON file with UTF-8 encoding
            with DATA_FILE.open('r', encoding='utf-8') as f:
                return json.load(f)
        except Exception:
            # Show error message if reading fails
            messagebox.showerror("Error", f"Failed to read {DATA_FILE}")
            # Return empty list on error
            return []
    # Method to write data to JSON file
    def write_json(self):
        try:
            # Open and write data to JSON file with pretty formatting
            with DATA_FILE.open('w', encoding='utf-8') as f:
                # Use indent=2 for readability, ensure_ascii=False for special characters
                json.dump(self.data, f, indent=2, ensure_ascii=False)
        except Exception:
            # Show error message if writing fails
            messagebox.showerror("Error", f"Failed to write {DATA_FILE}")
    # ========== DATA LOADING METHODS ==========
    # Method called when Load button is clicked or data changes
    def on_load(self):
        # Read data from JSON file
        self.data = self.read_json()
        # Ensure every book has a unique id; if missing, create one and persist
        updated = False
        for b in self.data:
            if 'id' not in b:
                b['id'] = str(uuid.uuid4())
                updated = True
        if updated:
            self.write_json()
        # Update category dropdown with available categories
        self.populate_categories()
        # Refresh the table display
        self.refresh_treeview()
    # Method to populate category dropdown
    def populate_categories(self):
        # Extract unique categories from books and sort alphabetically
        cats = sorted({item.get('category','') for item in self.data if item.get('category')
})
        # Create list with "All" option first
        values = ["All"] + cats
        # Set dropdown values and attempt to preserve any previous selection
        self.category_cb['values'] = values
        current = self.category_var.get()
        if current and current in values:
```

```
            self.category_cb.set(current)
        else:
            self.category_cb.set("All")
        # If dataset is empty ensure combobox still has only "All"
        if not cats:
            self.category_cb['values'] = ["All"]
            self.category_cb.set("All")
    # ========== TABLE DISPLAY METHODS ==========
    # Method to refresh/update the table display
    def refresh_treeview(self, category=None, name_filter=None):
        # Clear all existing rows from table
        for r in self.tree.get_children():
            self.tree.delete(r)
        # Start with all data
        # If no category argument given, read current combobox selection
        if category is None:
            category = self.category_var.get() or "All"
        filtered = self.data

        # Filter by category if selected (not "All")
        if category and category != "All":
            filtered = [b for b in filtered if b.get('category') == category]

        # Filter by book name search query (case-insensitive)
        if name_filter:
            q = name_filter.strip().lower()
            filtered = [b for b in filtered if q in b.get('name','').lower()]
        # ========== SORTING ==========
        # Helper function to extract year from book data for sorting
        def get_year(b):
            try:
                # Try to convert date to integer
                return int(b.get('date') or 0)
            except Exception:
                # If conversion fails, return 0
                return 0
        # Sort filtered books by date (ascending or descending based on flag)
        filtered = sorted(filtered, key=get_year, reverse=not self.sort_ascending)
        # ========== INSERT ROWS INTO TABLE ==========
        # Add each book to table with alternating row colors and serial numbers
        for idx, b in enumerate(filtered):
            # Serial number (1-based index for visible rows)
            serial = idx + 1
            # Determine tag (color) for this row: even index = blue, odd index = white
            tag = 'evenrow' if idx % 2 == 0 else 'oddrow'
            # Ensure book has an id and use it as the tree item iid
            item_id = b.get('id') or str(uuid.uuid4())
            if 'id' not in b:
                b['id'] = item_id
            # Insert row with iid set to the book id
            self.tree.insert('', tk.END, iid=item_id, values=(serial, b.get('name',''), b.ge
t('author',''), b.get('date',''), b.get('category','')), tags=(tag,))
        # ========== SHOW/HIDE EMPTY STATE ==========
        # If no books, show the empty state message
        if not filtered:
            self.empty_label.pack(side=tk.TOP, fill=tk.BOTH, expand=True)
        else:
            # Otherwise hide it by removing from display
            self.empty_label.pack_forget()
    # ========== SEARCH FUNCTIONALITY ==========
    # Method called when Search button is clicked
    def on_search(self):
        # Get selected category from dropdown
        cat = self.category_var.get()
        # Get search query from textbox
        name = self.search_var.get()
        # Refresh table with both filters applied
        self.refresh_treeview(category=cat, name_filter=name)
    # Method called when the category combobox selection changes
    def on_category_change(self):
        # Read current category and refresh table to show only that category
        cat = self.category_var.get() or "All"
```

```
    self.refresh_treeview(category=cat, name_filter=self.search_var.get())
# ========== CONTEXT MENU HANDLERS ==========
# Show context menu on right-click
def on_right_click(self, event):
    # Identify row under mouse
    iid = self.tree.identify_row(event.y)
    if iid:
        # Select the row and show the menu
        self.tree.selection_set(iid)
        try:
            self.context_menu.tk_popup(event.x_root, event.y_root)
        finally:
            self.context_menu.grab_release()
# Called when Edit is selected from the context menu
def on_edit_selected(self):
    sel = self.tree.selection()
    if not sel:
        messagebox.showwarning("No selection", "Please select a book to edit")
        return
    vals = self.tree.item(sel[0])['values']
    # Use tree item's iid (book id) to find the book in data
    iid = sel[0]
    idx = None
    for i, b in enumerate(self.data):
        if str(b.get('id')) == str(iid):
            idx = i
            break
    if idx is None:
        messagebox.showerror("Not found", "Selected book was not found in data")
        return
    # Open the edit dialog for the selected book
    EditBookWindow(self, idx)
# ========== ADD BOOK FUNCTIONALITY ==========
# Method called when New button is clicked
def on_new(self):
    # Open the "Add New Book" dialog window
    NewBookWindow(self)
# ========== DELETE BOOK FUNCTIONALITY ==========
# Method called when Erase button is clicked
def on_erase(self):
    # Get the selected row from table
    sel = self.tree.selection()
    # If no row is selected, show warning and return
    if not sel:
        messagebox.showwarning("No selection", "Please select a book to delete")
        return

    # Get data (values) from selected row
    # Use selected item's iid to find book by id
    iid = sel[0]
    name = None
    # Find matching book index by id
    to_remove = None
    for i, b in enumerate(self.data):
        if str(b.get('id')) == str(iid):
            to_remove = i
            name = b.get('name')
            break

    # Ask user to confirm deletion with dialog
    if not messagebox.askyesno("Confirm", f"Delete '{name}'?"):
        return

    # If book found in data list by id, remove it

    # If book found in data list, remove it
    if to_remove is not None:
        del self.data[to_remove]
        # Save changes to JSON file
        self.write_json()
        # Reload and refresh the display
        self.on_load()
```

```
    # ========== SORT FUNCTIONALITY ==========
    # Method called when Sort button is clicked to toggle sort order
    def on_sort_toggle(self):
        # Toggle sort order (True becomes False, False becomes True)
        self.sort_ascending = not self.sort_ascending
        # Update button text to show current sort direction (↑ or ↓)
        arrow = '↑' if self.sort_ascending else '↓'
        self.sort_btn.config(text=f"Sort by Date {arrow}")
        # Refresh table with new sort order applied
        self.refresh_treeview(category=self.category_var.get(), name_filter=self.search_var.
get())
# ========== ADD BOOK WINDOW CLASS ==========
# Dialog window for adding a new book to the library
class NewBookWindow:
    # Constructor to create the dialog
    def __init__(self, app: LibraryApp):
        # Store reference to main app
        self.app = app
        # Create a new top-level window (dialog/popup)
        self.top = tk.Toplevel(app.root)
        # Set dialog window title
        self.top.title("Add New Book")
        # Set dialog window size (width x height)
        self.top.geometry("420x320")
        # Create frame to hold form elements with padding
        frm = ttk.Frame(self.top, padding=12)
        # Pack frame to fill the dialog
        frm.pack(fill=tk.BOTH, expand=True)
        # ========== BOOK NAME INPUT FIELD ==========
        # Label for book name field
        ttk.Label(frm, text="Name").grid(row=0, column=0, sticky='w')
        # Variable to store book name entered by user
        self.name_var = tk.StringVar()
        # Text entry field for book name (48 characters wide)
        ttk.Entry(frm, textvariable=self.name_var, width=48).grid(row=0, column=1, pady=6)
        # ========== AUTHOR INPUT FIELD ==========
        # Label for author field
        ttk.Label(frm, text="Author").grid(row=1, column=0, sticky='w')
        # Variable to store author name entered by user
        self.author_var = tk.StringVar()
        # Text entry field for author name (48 characters wide)
        ttk.Entry(frm, textvariable=self.author_var, width=48).grid(row=1, column=1, pady=6)
        # ========== YEAR INPUT FIELD ==========
        # Label for publication year field
        ttk.Label(frm, text="Date (year)").grid(row=2, column=0, sticky='w')
        # Variable to store publication year entered by user
        self.date_var = tk.StringVar()
        # Text entry field for year (20 characters wide)
        ttk.Entry(frm, textvariable=self.date_var, width=20).grid(row=2, column=1, sticky='w
', pady=6)
        # ========== CATEGORY INPUT FIELD ==========
        # Label for category field
        ttk.Label(frm, text="Category").grid(row=3, column=0, sticky='w', pady=6)
        # Variable to store selected category
        self.cat_var = tk.StringVar()
        # List of predefined categories available for selection
        cats = ["Book", "Film", "Magazine"]
        # Dropdown combobox for category selection (editable, 45 characters wide)
        self.cat_cb = ttk.Combobox(frm, textvariable=self.cat_var, values=cats, width=45)
        # Pack combobox in grid at row 3, column 1
        self.cat_cb.grid(row=3, column=1, sticky='ew', pady=6)
        # Set "Book" (index 0) as default category
        self.cat_cb.current(0)
        # ========== SAVE BUTTON ==========
        # Button to save the new book to the library
        save_btn = ttk.Button(frm, text="Save", command=self.on_save)
        # Place button at row 4, column 1 with top padding
        save_btn.grid(row=4, column=1, pady=(18, 0))
    # ========== SAVE FUNCTIONALITY ==========
    # Method called when Save button is clicked
    def on_save(self):
        # Get book name and remove extra whitespace from both ends
```

```
        name = self.name_var.get().strip()
        # Get author name and remove extra whitespace
        author = self.author_var.get().strip()
        # Get publication year and remove extra whitespace
        date = self.date_var.get().strip()
        # Get selected category, use 'Uncategorized' if empty
        cat = self.cat_var.get().strip() or 'Uncategorized'

        # Validate that book name is provided (required field)
        if not name:
            messagebox.showwarning("Validation", "Name is required")
            return

        # Create dictionary with book data and a unique id
        new = {"id": str(uuid.uuid4()), "name": name, "author": author, "date": date, "categ
ory": cat}
        # Add new book dictionary to the data list
        self.app.data.append(new)
        # Save updated data list to JSON file
        self.app.write_json()
        # Reload app to show new book in the library
        self.app.on_load()
        # Close this dialog window
        self.top.destroy()
# ========== EDIT BOOK DIALOG ==========
class EditBookWindow:
    # Constructor to create the edit dialog for an existing book
    def __init__(self, app: LibraryApp, index: int):
        self.app = app
        self.index = index
        # Create top-level window for editing
        self.top = tk.Toplevel(app.root)
        self.top.title("Edit Book")
        self.top.geometry("420x320")
        frm = ttk.Frame(self.top, padding=12)
        frm.pack(fill=tk.BOTH, expand=True)
        # Load current book data
        book = app.data[index]
        # Name field
        ttk.Label(frm, text="Name").grid(row=0, column=0, sticky='w')
        self.name_var = tk.StringVar(value=book.get('name',''))
        ttk.Entry(frm, textvariable=self.name_var, width=48).grid(row=0, column=1, pady=6)
        # Author field
        ttk.Label(frm, text="Author").grid(row=1, column=0, sticky='w')
        self.author_var = tk.StringVar(value=book.get('author',''))
        ttk.Entry(frm, textvariable=self.author_var, width=48).grid(row=1, column=1, pady=6)
        # Date field
        ttk.Label(frm, text="Date (year)").grid(row=2, column=0, sticky='w')
        self.date_var = tk.StringVar(value=book.get('date',''))
        ttk.Entry(frm, textvariable=self.date_var, width=20).grid(row=2, column=1, sticky='w
', pady=6)
        # Category field
        ttk.Label(frm, text="Category").grid(row=3, column=0, sticky='w', pady=6)
        self.cat_var = tk.StringVar(value=book.get('category','Uncategorized'))
        cats = ["Book", "Film", "Magazine"]
        self.cat_cb = ttk.Combobox(frm, textvariable=self.cat_var, values=cats, width=45)
        self.cat_cb.grid(row=3, column=1, sticky='ew', pady=6)
        if self.cat_var.get() not in cats:
            self.cat_cb.set(self.cat_var.get())
        else:
            self.cat_cb.current(cats.index(self.cat_var.get()))
        # Save button
        save_btn = ttk.Button(frm, text="Save", command=self.on_save)
        save_btn.grid(row=4, column=1, pady=(18, 0))
    # Save changes back to app.data and persist
    def on_save(self):
        name = self.name_var.get().strip()
        author = self.author_var.get().strip()
        date = self.date_var.get().strip()
        cat = self.cat_var.get().strip() or 'Uncategorized'
        if not name:
            messagebox.showwarning("Validation", "Name is required")
```

```
            return
        # Update the book in-place while preserving its unique id
        old = self.app.data[self.index]
        self.app.data[self.index] = {**old, 'name': name, 'author': author, 'date': date, 'c
ategory': cat}
        # Persist changes and refresh
        self.app.write_json()
        self.app.on_load()
        # Close dialog
        self.top.destroy()
# ========== MAIN FUNCTION ==========
# Entry point of the application
def main():
    # Create the root window (main application window)
    root = tk.Tk()

    # ========== APPLY MODERN THEME ==========
    # Try to use modern 'clam' theme if available
    try:
        style = ttk.Style()
        # Set theme to 'clam' for modern appearance
        style.theme_use('clam')
    except Exception:
        # Use default theme if 'clam' not available
        pass
    # Create and initialize the main app
    LibraryApp(root)
    # Start the GUI event loop (keeps window open and responsive)
    root.mainloop()
# ========== SCRIPT ENTRY POINT ==========
# This condition runs when the script is executed directly (not imported as a module)
if __name__ == '__main__':
    # Call main function to start the application
    main()
```